API Documentation

Transaction Ledger API

Overview

- Base URL: http://127.0.0.1:8000
- Content type: application/json
- Authentication: not enabled in this assessment build; deploy behind an authenticated, trusted boundary before exposing it publicly.
- Currency: amounts are exact decimal values with two fractional digits. The UI displays USD; no currency conversion is performed.

Root and API index

GET / and GET /api/ return a JSON index describing the API and its primary routes. Unknown paths below /api/ return the standard 404 error envelope.

```
{
  "service": "Transaction Ledger API",
  "api_base": "http://127.0.0.1:8000/api/",
  "endpoints": {
    "dashboard": "/api/dashboard/",
    "accounts": "/api/accounts/",
    "transactions": "/api/transactions/",
    "transfers": "/api/transfers/"
  }
}
```

All errors use this envelope:

```
{
  "error": {
    "code": "validation_error",
    "message": "The request contains invalid data.",
    "details": {
      "amount": ["Amount must be greater than zero."]
    }
  } }
```

details is null for simple request errors and an object for field validation errors.

Endpoints

GET /dashboard/

Return server-computed ledger totals, six monthly credit/debit buckets, and the five latest entries. Monetary values are returned as decimal strings; the chart converts its display-only values for plotting.

Success — 200 OK:

```
{
  "account_count": 0,
  "total_balance": "0.00",
  "total_credits": "0.00",
  "total_debits": "0.00",
  "transaction_count": 0,
  "cash_flow": [
    { "month": "May", "credits": "0.00", "debits": "0.00" },
    { "month": "Jun", "credits": "0.00", "debits": "0.00" },
    { "month": "Jul", "credits": "0.00", "debits": "0.00" },
```

```
    { "month": "Aug", "credits": "0.00", "debits": "0.00" },
    { "month": "Sep", "credits": "0.00", "debits": "0.00" },
    { "month": "Oct", "credits": "0.00", "debits": "0.00" }
  ],
  "recent_transactions": []
}
```

cashflow always contains six entries, from five months before the current month through the current month. Each recent transaction has the same shape as the history transaction object.

GET /transactions/

Search/filter/paginate across all accounts. This feed is used by the workspace-wide transaction page; account-specific history remains available below.

Query parameters: optional accountid (existing account UUID), optional type (credit or debit), optional search (case-insensitive substring in account holder, description, or reference ID), optional sort (newest, oldest, amountdesc, amountasc; defaults to newest), and the same page/limit rules as account history.

Success — 200 OK: the same pagination envelope and transaction object shape as GET /accounts/{accountid}/transactions/.

Errors: 400 Bad Request for invalid filters, invalid/missing account UUIDs, unsupported sort values, or invalid pagination.

GET /accounts/

List accounts, newest first. Returns an array; an empty ledger returns [].

Success — 200 OK:

```
[
  {
    "id": "560c5aa3-4f07-42bd-a9d0-088fcade5169",
    "holder_name": "Mira Patel",
    "balance": "125.00",
    "created_at": "2025-01-15T10:30:00Z"
  }
]
```

POST /accounts/

Create an account. holdername is required. The server generates the UUID and timestamp; balance always starts at 0.00. Sending balance is rejected.

Request:

```
{ "holder_name": "Mira Patel" }
```

Success — 201 Created: one account object as shown above, with "balance": "0.00".

Errors: 400 Bad Request for missing/blank holder name or a supplied balance.

GET /accounts/{accountid}/balance/

Get the current balance for a UUID account ID.

Success — 200 OK:

```
{ "account_id": "560c5aa3-4f07-42bd-a9d0-088fcade5169", "balance": "125.00" }
```

Errors: 404 Not Found if the account does not exist; 400 Bad Request for a malformed account UUID.

POST /accounts/{accountid}/transactions/

Record a credit or debit. The account ID comes from the URL and cannot be changed in the body.

Request:

```
{
  "type": "credit",
  "amount": "125.00",
  "description": "Invoice payment",
  "reference_id": "INV-2025-001"
}
```

type must be credit or debit; amount must be greater than zero and have at most two decimal places; referenceid is required (maximum 100 characters); description is optional (maximum 255 characters).

New transaction — 201 Created:

```
{
  "transaction": {
    "id": "345a26b4-9f2d-4c79-8fd8-50a2ce5a34de",
    "account_id": "560c5aa3-4f07-42bd-a9d0-088fcade5169",
    "type": "credit",
    "amount": "125.00",
    "description": "Invoice payment",
    "reference_id": "INV-2025-001",
    "created_at": "2025-01-15T10:30:00Z"
  },
  "duplicate": false
}
```

Debit transactions subtract the amount. An insufficient balance or a credit that would exceed the account balance precision returns 409 Conflict and does not create a transaction. An account ID that does not exist returns 404 Not Found. **Duplicate transaction behavior**

referenceid is unique per account. Repeating a reference on the same account returns 200 OK, the existing transaction, and "duplicate": true; it does not change the balance. A different account may use that same reference and its own balance is updated.

GET /accounts/{accountid}/transactions/

Get an account's transaction history, newest first (creation timestamp descending, UUID as a stable tiebreaker).

Query parameters:

- type: optional credit or debit; omitted returns both.
- page: positive integer; defaults to 1.
- limit: positive integer; defaults to 10, maximum 100.
- sort: optional for the global feed; defaults to newest-first. Values are newest, oldest, amountdesc, or amountasc.

Success — 200 OK:

```
{
  "current_page": 1,
  "page_size": 10,
  "total_records": 1,
  "total_pages": 1,
  "results": [
    {
      "id": "345a26b4-9f2d-4c79-8fd8-50a2ce5a34de",
      "account_id": "560c5aa3-4f07-42bd-a9d0-088fcade5169",
      "type": "credit",
      "amount": "125.00",
      "description": "Invoice payment",
```

```
      "reference_id": "INV-2025-001",
      "created_at": "2025-01-15T10:30:00Z"
    }
  ]
}
```

Errors: 400 Bad Request for invalid type/sort, non-positive/non-integer page or limit, limit above 100, or a page beyond the available range; 404 Not Found for a missing account.

Transactions are immutable. There are no update or delete endpoints.

POST /transfers/

Atomically transfer money between two different existing accounts. The source must have sufficient balance. A transfer writes one debit on the source and one credit on the destination using the same reference ID. Both accounts are locked in a stable order where the database supports row locks.

Request:

```
{
  "source_account_id": "560c5aa3-4f07-42bd-a9d0-088fcade5169",
  "destination_account_id": "fd7d0cd0-23cd-4ec4-8db9-2fd0565a5b18",
  "amount": "25.50",
  "reference_id": "TRF-2025-001",
  "description": "Monthly allocation"
}
```

New transfer — 201 Created:

```
{   "source_transaction": { "id": "…", "account_id": "…", "type": "debit", "amount": "25.50",
"description":
     "Monthly allocation", "reference_id": "TRF-2025-001", "created_at": "…" },
  "destination_transaction": { "id": "…", "account_id": "…", "type": "credit", "amount": "25.50",
"description": "Monthly allocation", "reference_id": "TRF-2025-001", "created_at": "…" },
  "duplicate": false
}
```

Repeating a transfer reference on the same source account returns 200 OK with "duplicate": true only when the existing debit and matching destination credit have the same amount. Reusing the reference for a different amount, another destination, or a non-transfer transaction returns 409 Conflict. References also must be unused on the destination account to satisfy the account-scoped uniqueness constraint. Insufficient funds, a destination balance limit, or a reference conflict returns 409 Conflict; invalid/same account IDs or missing accounts return 400 Bad Request. No partial transfer is committed.

HTTP status summary

Status	Meaning
200	Read succeeded or idempotent duplicate request
201	Account, transaction, or transfer created
400	Invalid request, invalid pagination, or missing transfer account
404	Account not found or unmatched route
409	Insufficient funds, balance limit exceeded, or transfer reference conflict

SQLite is used for local development and tests and does not implement row-level SELECT ... FOR UPDATE locking. Use MySQL with InnoDB and validate concurrent posting behavior there before deploying a workload that depends on row locks. The database also enforces nonnegative account balances, the supported balance ceiling, positive transaction amounts, valid transaction types, and unique per-account references.